Channel Estimation for Rydberg Receivers Using Learned Hankel Filtering

Abdullah Haatim

***Abstract*—Rydberg receivers use excited atoms to sense radio signals. In a magnitude-only receiver, the measured signal strength does not directly reveal the wireless channel. Recovering the channel is difficult because the measurements contain noise and do not directly provide phase. Channel structure can help, but a fixed structural rule may also remove useful information. This paper studies a small learning-based estimator that adjusts the amount of Hankel filtering using the current measurements. It alternates between fitting the receiver observations and filtering the channel estimate. Training uses a withheld part of the observations, without true channels as targets. In simulations, the method gives lower average channel error and shorter processing time than the tested classical Hankel estimator. Further tests show that using both channel-structure information and noise-related information improves average accuracy. The benefit is not uniform: the classical method remains better with a weak reference signal and in some channels with few paths. The results support learned Hankel filtering as a useful option for the tested receiver model, while showing the need to state its operating limits clearly.**

***Index Terms*—Channel estimation, Hankel filtering, magnitude measurements, Rydberg receivers, self-supervised learning.**

I. Introduction

RYDBERG atoms are atoms in highly excited states that can respond to radio-frequency electric fields. This makes them useful for studying new ways to receive wireless signals. In the receiver model considered here, signals from several users are combined with a known reference signal. The receiver measures the magnitude of that combined signal, rather than its full complex value. The wireless channel describes how each user's signal changes in strength and phase before reaching the receiver. Estimating it is an important step in recovering the transmitted information. Both model-based and learning-based methods have been developed for this problem, including URformer [1], [2].

Signals arriving through a small number of paths often form a repeated pattern across a regular receiver array. A Hankel matrix arranges the channel samples so that this pattern can be used for estimation. However, a channel does not always follow a simple structural model closely. Filtering too strongly can remove real channel details, while filtering too weakly can leave noise in the estimate. Trying many possible structural settings also takes time. This motivates a small model that learns how much filtering to apply from the available measurements, without requiring the true channel during training.

A. Haatim is with the Department of Electrical and Electronics Engineering, Birla Institute of Technology and Science, Pilani, India (e-mail: f20220519@pilani.bits-pilani.ac.in).

Research draft, September 23, 2026. Results are simulation only.

The contributions of this paper are:

1) A 188-parameter estimator that combines Rician measurement updates with smooth Hankel filtering. A shared controller adjusts the filtering strength using the estimated channel spectrum, residual and known noise level.
2) An application of held-out-measurement training to this estimator. True channels are not training targets or checkpoint-selection scores. The contribution is the receiver-specific estimator, not a new principle of self-supervised learning.
3) Paired simulation comparisons of accuracy, processing time and simpler versions of the method, including fresh-test replication and input-removal tests. Weak-reference and few-path failures are reported.

II. Related Work

Learning without clean target signals is already used in reconstruction. One approach uses part of the measurements to form an estimate and another part to train the model [3]. Self-supervised learning has also been used to remove noise from Rydberg sensor signals [4]. Huang et al. learn Hankel-factor updates from fully sampled signal targets [5]. Luo et al. combine self-supervision and structured low-rank reconstruction for linearly sampled complex MRI data [6]. DeepTensor represents low-rank factors by networks fitted to the observed object [7]. Rician-likelihood training for magnitude data is also established [8]. Thus neither self-supervision, learned Hankel processing nor the Rician loss is new on its own.

Our focus is a particular estimation task: recovering a complex multiuser channel from noisy magnitudes with a known reference. We combine the receiver's noise model with a small network that controls smooth Hankel filtering. Our companion manuscript studies classical Hankel estimation and rank selection [9]. Here, the emphasis is on learning filtering strength without channel labels and measuring accuracy and processing cost. Unlike factor-generating networks, our controller sets two filtering controls; a fixed ten-stage estimator performs the channel inversion. Unlike methods that restore measured complex samples directly, it fits noisy magnitudes through a known reference. These are design differences, not a claim of superiority over those methods. Self-supervised low-rank channel estimation in XL-MIMO is also listed in the CCAI 2026 program [10]. Its full method was not available for a detailed comparison. We make no first-ever claim and do not claim to reproduce the published URformer results.

III. Receiver and Channel Model

Consider a receiver array with N elements and K users. Each user sends P known training symbols, called pilots.

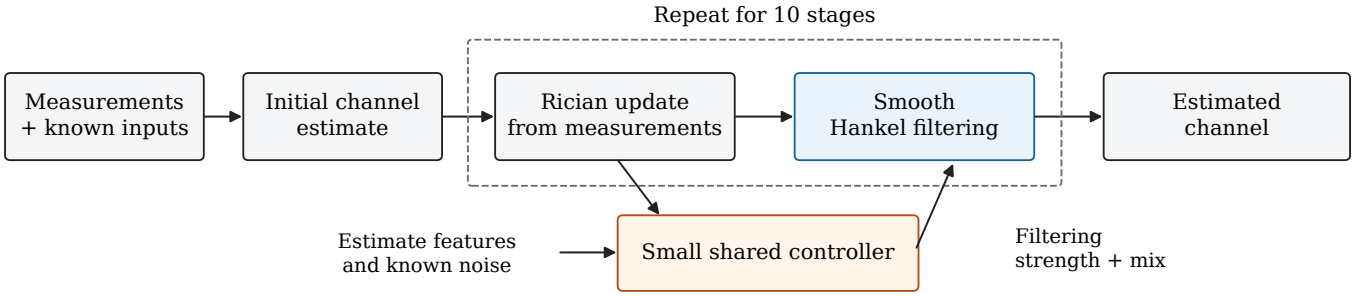


Fig. 1. Estimator overview. The shared controller uses the current estimate and known receiver information to choose filtering strength and mixing. During training, the nine withheld pilots are used only to score predictions; they do not enter initialization or the ten estimation stages.

These pilots allow the receiver to estimate the channel before detecting unknown data. The observations are

$$Z = |GS + B + W|, \qquad W_{np} \sim \mathcal{CN}(0, v), \tag{1}$$

where $G \in \mathbb{C}^{N\times K}$ is the unknown channel, $S \in \mathbb{C}^{K\times P}$ contains the pilots, and $B \in \mathbb{C}^{N\times P}$ is the known reference. Noise samples are independent. The magnitude acts on each entry, so Z is nonnegative and does not directly measure phase. The noise variance v is the total complex variance; each real component has variance $v/2$.

The reference makes the measured magnitudes depend on the relative phases of the user signals. It does not guarantee that every channel can be recovered accurately. Reference calibration and noise variance are assumed known, except in tests that deliberately supply an incorrect noise variance. Receiver conversion gain is absorbed into the model's scaling. These are simulation assumptions, not claims about all physical Rydberg receivers.

The main channel uses a half-wavelength uniform linear array:

$$g_{nk} = \sum_{\ell=1}^{L_k} \alpha_{\ell k} e^{-j\pi n \sin\theta_{\ell k}}, \quad n = 0, \dots, N-1. \tag{2}$$

Each path has gain $\alpha_{\ell k}$ and arrival angle $\theta_{\ell k}$. Angles are uniform on $[-\pi/2, \pi/2]$, and independent gains follow $\mathcal{CN}(0, 1/L_k)$, giving unit average power per user. Pilot entries follow $\mathcal{CN}(0, 1)$. The reference has a known spatial response and constant commanded waveform across the pilot observations.

SNR measures total average user power relative to noise. RSR measures reference power relative to one user's average power. For the unit-power users used here,

$$\mathrm{SNR} = 10\log_{10}(K/v), \quad \mathrm{RSR} = 10\log_{10}|\alpha_b|^2, \tag{3}$$

where α_b is the reference amplitude. With three users, an RSR of 12 dB is about 7.23 dB relative to their combined average signal power. These definitions must remain separate when interpreting the results.

IV. Proposed Estimator

A. Updating the Estimate from Measurements

The estimator starts from a channel estimate formed from the available observations, pilots and reference. It repeats a measurement update and a Hankel-filtering step ten times, as illustrated in Fig. 1.

The measurement update uses the Rician distribution, which describes the magnitude of a complex signal corrupted by the assumed Gaussian noise. For signal magnitude $a = |\mu|$, its density is

$$p(z \mid a, v) = \frac{2z}{v} \exp\left(-\frac{z^2 + a^2}{v}\right) I_0(2za/v). \tag{4}$$

Here I_0 is a modified Bessel function. This model accounts for noise acting before the magnitude is taken. It does not replace the magnitude observations with an additive Gaussian-noise approximation.

Let $M = GS + B$ and $\kappa = 2Z|M|/v$, with entrywise operations in

$$Q = Z\frac{I_1(\kappa)}{I_0(\kappa)}\frac{M}{|M|}. \tag{5}$$

I_1 is the first-order modified Bessel function. The continuous limit is used at $M = 0$. The expectation-maximization (EM) update is

$$G_{\mathrm{EM}} = (Q - B)S^H(SS^H)^{-1}, \tag{6}$$

$$G_{\mathrm{dc}} = G + \gamma_t(G_{\mathrm{EM}} - G), \tag{7}$$

where $\gamma_t = 2\,\mathrm{sigmoid}(a_t)$ is a learned step size at stage t. The inverse is evaluated by a linear solve. The input pilot matrix must have full row rank. The implementation requires at least $2K$ input pilots; this is a numerical design rule, not a proof of unique channel recovery.

B. Smooth Hankel Filtering

Each user's channel vector in G_{dc} is arranged into a balanced Hankel matrix $H = \mathcal{H}(g_{\mathrm{dc},k})$, whose entries are constant along antidiagonals. At $N = 32$, its size is 17×16. Its singular values describe the strengths of different matrix components. Their collection is called the singular-value spectrum.

A hard rank rule keeps a chosen number of components and discards the rest. We instead reduce each singular value smoothly:

$$\widehat{\sigma}_i = \frac{\sigma_i^3}{\sigma_i^2 + \tau^2}. \tag{8}$$

Larger τ gives stronger filtering. Small singular values are reduced by a larger fraction than large ones. This does not mean that every small component is noise: some may contain real channel information.

Writing $q = \lceil N/2 \rceil$ and $s_H = \|H\|_F/\sqrt{q}$, the controller output u sets

$$\tau = s_H\big[0.001 + \mathrm{softplus}(u)\big]. \tag{9}$$

The floor makes the normalized solve nonsingular. The equivalent matrix operation is

$$\widehat{H} = H - \tau^2 H(H^H H + \tau^2 I)^{-1}. \tag{10}$$

The code normalizes by s_H before solving and uses a small scale guard at zero. Gradients pass through this solve without an identity-backward approximation. Antidiagonal averaging, denoted by $\mathcal{H}^\dagger$, converts the filtered matrix back to a channel vector:

$$g_k^{t+1} = g_{\mathrm{dc},k} + \eta_{t,k}\left(\mathcal{H}^\dagger \widehat{H}_k - g_{\mathrm{dc},k}\right). \tag{11}$$

The learned mixing factor $\eta_{t,k} = \mathrm{sigmoid}(w)$ lies between zero and one. Neither η nor τ is an estimate of the true path count. The complete learned iteration is not guaranteed to reduce channel error at every stage or to converge.

C. Choosing the Filtering Strength

A shared network with eight inputs, 16 hidden units and two outputs chooses u and w. It uses a SiLU activation, $x\,\mathrm{sigmoid}(x)$. The same network is used for every user and stage. It has 178 parameters; ten learned step sizes bring the total to 188.

The first three inputs describe the Hankel spectrum. If $p_i = \sigma_i/\sum_j \sigma_j$, they are

$$\frac{\exp(-\sum_i p_i \log p_i)}{q}, \qquad p_1, \qquad \sum_{i=q-\lceil q/2\rceil+1}^{q} p_i. \tag{12}$$

These describe how spread out the components are, the largest component's share, and the smaller components' combined share.

Two inputs describe noise and the difference between measured and predicted magnitudes, called the measurement residual:

$$f_4 = \log\left(1 + \frac{\mathrm{mean}[(Z - |G_{\mathrm{dc}}S + B|)^2]}{v}\right), \tag{13}$$

$$f_{5,k} = \log\left(1 + \frac{v\,\mathrm{tr}[(SS^H)^{-1}]}{K\widehat{p}_k}\right), \quad \widehat{p}_k = \frac{\|g_{\mathrm{dc},k}\|^2}{N}. \tag{14}$$

The remaining inputs are $P/(2K)$, $\log q$ and $(t+1)/10$, with $t = 0, \ldots, 9$. Small denominator guards prevent division by zero. The model does not receive true channels, true path counts or channel-family labels. Its outputs change with the observations, but its learned weights remain fixed during use.

TABLE I
MAIN SIMULATION AND TRAINING SETTINGS

Setting	Value
Array elements / users / pilots	$32/3/30$
Training / validation cases	16,384/2,048
SNR values (dB)	$-10, -5, 0, 5, 10, 15, 20$
Training RSR values (dB)	$5, 12, 20$
Training seeds	3, sharing training cases
Estimator stages / parameters	$10/188$
Training epochs / batch size	$20/32$
Optimizer / learning rate	Adam / 0.001
Input / withheld pilots	$21/9$
Training / evaluation precision	Single / double

D. Training Without True Channels

Each training example has 30 pilot columns. A random split gives 21 columns to the estimator and withholds nine. The estimate is used to predict the withheld magnitudes. Training minimizes their mean negative log likelihood, computed stably as

$$\ell(z, \mu, v) = \frac{(z - |\mu|)^2}{v} - \log I_{0e}(2z|\mu|/v), \tag{15}$$

where I_{0e} is the exponentially scaled Bessel function. An observation-only term is omitted because it is the same for the models being compared. Small-field series evaluate the continuous limits stably. The loss fits the amplitude parameter $|\mu|$ of the Rician density, not its mean; these are different in noise.

This is self-supervised learning: the observations provide training information without a true-channel target. Splits change each epoch, which is one pass through the training examples. Separate validation examples and a fixed split choose the saved model. At deployment, all available pilots are used, with no extra measurements and no further weight training. This changes the input pilot count from its training value, so performance under the change is tested rather than assumed.

V. SIMULATION STUDY

A. Setup and Error Measure

Table I lists the main settings. Half the training examples have random counts of three to seven paths per user. The rest are spread over fixed counts of 2, 4, 8, 12 and 16. Validation gives equal weight to 126 SNR, RSR and path-setting combinations. The simulated condition labels, including path settings, are used only to set these validation weights; they are not estimator inputs. Model selection uses held-out likelihood, not channel error.

The main study trains 18 models: two update choices, three filtering choices and three training seeds. Seeds change initial weights, batch order and pilot splits, but share the same training cases. EM remains the primary update because it was selected by validation likelihood before test generation. The 18-model training campaign took about 3.67 hours of elapsed time with overlapping workers, excluding data preparation and preflight. This is a campaign cost, not the training time of one model.

The main evaluation covers 32 conditions with 400 paired cases per condition. Two noise-input tests reuse cases from another condition, giving 12,000 distinct cases. Methods receive the same channel, pilots and noise within each case. The main classical comparator is HS-GS, the Hankel-structured iterative estimator with rank search from the companion study [9]. For each case, HS-GS fits ranks $1, \ldots, q$ on 70% of the pilots using 25 EM updates and four alternating rank/Hankel projection sweeps per update. It selects one rank using squared magnitude residuals on the remaining pilots, then refits using all pilots for 100 updates. The rank search consumes no extra observations and is included in timing. Its selection loss differs from our Rician training loss. Other controls include fixed-iteration EM, Rician and amplitude-gradient methods, hard Hankel filtering and learned versions with global or no filtering. The global control learns a relative threshold and mixing factor per stage; its absolute threshold still scales with s_H. It has 30 parameters. The no-filtering model has ten. The hard-Hankel Rician-gradient control uses rank ten, selected on development likelihood, and one projection sweep per update. Constant-output controls replace the adaptive network's outputs with validation averages; they are interventions, not separately trained models.

Channel accuracy is measured by normalized mean squared error (NMSE):

$$\mathrm{NMSE}_{\mathrm{dB}} = 10 \log_{10} \frac{\sum_i \|\widehat{G}_i - G_i\|_F^2}{\sum_i \|G_i\|_F^2}. \tag{16}$$

Lower values are better. We average the per-seed dB scores, not the predicted channels. The main average gives equal weight to the seven SNR settings. A positive gain means that the proposed method has lower error than the comparator. Severe errors have per-case linear NMSE above one.

Uncertainty intervals use 4,000 paired resamples of whole test cases. Main-study intervals are pointwise 95% intervals, not simultaneous guarantees across every tested condition. They describe test-case variation for the saved models, not the full uncertainty from new training datasets. Stopping-controlled classical methods have a 1,000-update cap; reaching that cap is not treated as convergence.

B. Main Accuracy and Processing Time

Table II includes every main-study comparator. HS-GS is the strongest tested classical method on the declared SNR average. The secondary adaptive gradient-descent (GD) core has a slightly lower test NMSE than the primary EM core. We retain EM because core selection was made from validation likelihood, not from this test ranking. The proposed method is therefore not the best row on every metric.

Figure 2 compares the proposed method with HS-GS and simpler learned controls. Its average gain over HS-GS is 1.675 dB, with an interval of $[1.618, 1.730]$ dB. The three seed-specific gains range from 1.533 to 1.787 dB. The improvement is not equally strong at every SNR. At 20 dB, the gain is 0.139 dB and its interval, $[-0.017, 0.294]$ dB, includes zero. That condition therefore does not establish a clear improvement.

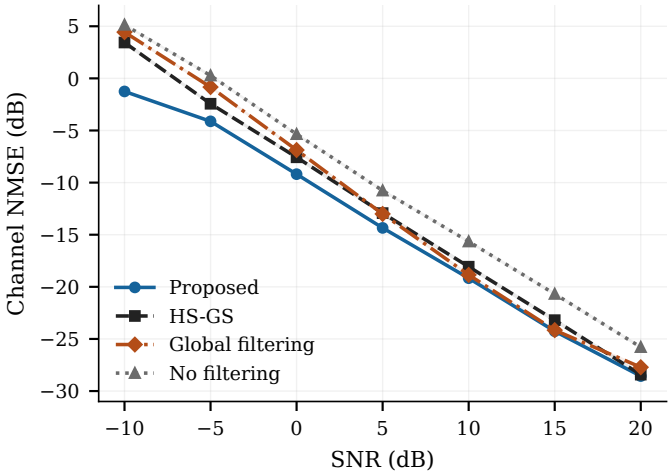


Fig. 2. Main SNR comparison at $N = 32$, $K = 3$, $P = 30$ and RSR 12 dB, with random three-to-seven-path channels. Lower NMSE is better. Lines show mean per-seed scores; shaded bands are conditional paired-world 95% intervals, not training-dataset uncertainty.

TABLE II
ALL MAIN-STUDY COMPARATORS

Method	NMSE (dB)	Time (ms)	Severe (%)
EM adaptive (primary)	-14.42	8.24	0.45
EM learned steps only	-10.41	2.98	22.11
EM global filtering	-12.43	5.61	19.19
EM constant-output control	-9.19	5.61	11.42
GD adaptive (secondary)	-14.59	8.00	0.85
GD learned steps only	-10.44	2.73	21.15
GD global filtering	-12.57	5.38	18.37
GD constant-output control	-9.95	5.36	12.36
HS-GS, rank search	-12.74	525.62	14.25
EM, 10 updates	-10.23	4.10	20.64
EM, 100 updates	-10.28	17.01	23.57
Rician GD, 10 updates	-9.31	4.17	17.86
Rician GD, 100 updates	-10.29	17.40	23.50
Rician GD, stopping rule	-10.27	45.37	23.75
Amplitude GD, 10 updates	-8.80	3.29	20.21
Amplitude GD, 100 updates	-9.65	8.87	26.96
Amplitude GD, stopping rule	-9.61	20.93	27.11
Rician GD, hard Hankel	-11.06	47.39	20.93
Linearized least squares	-6.92	1.06	19.86

NMSE and severe-error frequency are equal-weight averages across the seven primary SNR conditions. Time is the median at SNR 5 dB, RSR 12 dB, not across the SNR sweep. Learned scores average three seeds; classical cases are not counted three times. All numbers describe the tested implementations.

Figure 3 shows processing times measured under the same timing protocol. The proposed estimator takes a median of 8.24 ms per case, compared with 525.62 ms for HS-GS, a measured speedup of about 64 times. This does not establish the same speed difference on other hardware or against differently optimized software. Offline training is a separate cost and is not included in inference time. The learned implementations use PyTorch, while the classical ones use NumPy/SciPy. The no-filtering learned model is faster still, but has higher average error. The primary method's severe-error frequency is 0.45%, compared with 14.25% for HS-GS across the primary SNR cells. These are descriptive rates for this mixture, not guarantees of reliability for other operating conditions.

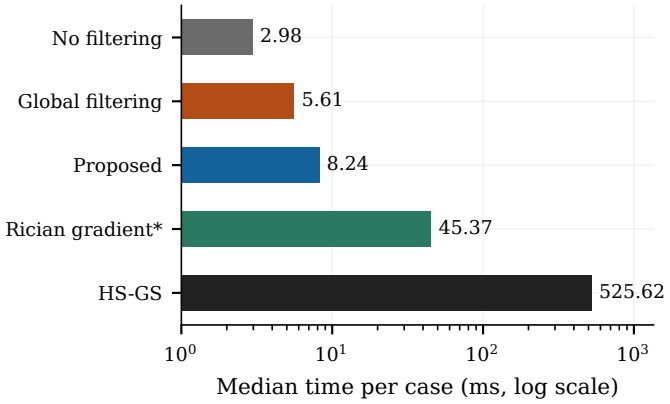


Fig. 3. Median processing time at SNR 5 dB and RSR 12 dB. Timing includes initialization and rank search where required. One-thread Apple M2 CPU, double precision, one case at a time, 100 cases and three repeats. The asterisk denotes a stopping-controlled Rician gradient solver. These are implementation-specific timings, not hardware-independent speed limits.

C. Operating Limits

Figure 4 includes settings where HS-GS remains better. The proposed method loses 1.617 dB at RSR 0 dB, 1.104 dB with two paths, and 0.203 dB with three paths. These are important limits. A model chosen for good average performance need not be best for a very simple channel or a weak reference. Using the correct noise distribution does not guarantee that the estimation problem is easy to solve.

Separate transfer tests give gains for the tested clustered, ten-path and unequal-power channels, and for supplied noise variances that are wrong by ± 3 dB. Results also change with array size and user count; not every condition has a clear advantage. Some path counts occur in training, so the sweep is not evidence of success on wholly unfamiliar channel types. These tests do not establish that the method resolves failures observed in earlier experiments with different protocols. The alternative-channel tests use four clusters of ten rays within $\pm 5^\circ$, ten independent equal-power paths, or ten paths with geometrically decreasing powers. Each alternative is rescaled to the corresponding base channel's total energy. They are one-condition stress tests at SNR 5 dB and RSR 12 dB, not full channel-family SNR sweeps.

D. Further Comparisons

A separate follow-up trains 15 models and uses two fresh test sets, each with seven main SNR settings and two difficult settings of 200 cases. Six primary comparisons are adjusted for multiple testing within each set. Table III reports the average gains. The independently generated training datasets each use one fixed training seed, so they provide limited replication rather than a full study of dataset and initialization variation.

The supervised comparison uses the same architecture, input pilots and training budget, but uses true channels as targets. Its training objective is mean linear channel NMSE. The label-free model has about a 1 dB advantage under the main averaging rule. However, selecting the supervised checkpoint by held-out likelihood reduces this to about 0.73 dB. Pooling linear errors across SNRs reduces it to about 0.36 dB.

TABLE III
FOLLOW-UP AVERAGE GAINS (DB)

Comparison	Set 1	Set 2
New-data adaptive vs. global filtering	2.252	2.185
New-data adaptive vs. no filtering	4.250	4.191
New-data adaptive vs. HS-GS	1.939	1.837
Original adaptive vs. supervised recipe	1.015	1.012
Original adaptive vs. final-only filtering	0.697	0.694
Fixed-open mix vs. original adaptive	0.064	0.066

Supervision is better in some settings, including the two-path test. It also transfers less consistently from 21 input pilots to 30 in a validation replay. We therefore do not conclude that self-supervision is generally better than supervision.

Filtering throughout the estimator improves average accuracy by about 0.70 dB over applying filtering only at the end. It also costs more: a separate timing sample gives about 4.65 ms and 2.05 ms, respectively. These timings are not combined with the main study's measurements to form a different speedup claim.

The learned mixing factor is usually close to one. Fixing it at one gives only a small average gain and worsens two of three training seeds on the two-path cases. The tested fixed-mix model still stores 188 parameters. We keep the original method and do not describe the mixing factor as a reliable decision to switch filtering on or off.

E. Which Controller Inputs Help?

A smaller study compares the full controller with spectrum-only, noise/residual-only and size/stage-only versions. Size and stage inputs are retained in every version. All versions use the same measurement update and Hankel filter. Each is trained from scratch on the same 4,096 development cases, with three seeds and 12 epochs. Two fresh test sets contain 2,304 cases in total: seven main SNR settings and two difficult settings, with 128 cases per condition in each set.

Figure 5(a) shows average gains of 1.22–1.25 dB over spectrum-only inputs, 0.57–0.58 dB over noise/residual-only inputs, and 2.21–2.23 dB over size/stage-only inputs. Gains are positive for all three seeds in both sets. Intervals adjusted for the three comparisons remain above zero. Final and validation-selected checkpoints give the same scores in this study.

The full controller is not better everywhere. In Fig. 5(b), it is about 0.39–0.40 dB worse than the noise/residual-only version at 5 dB SNR. It therefore fails the predeclared requirement that no main condition should worsen by more than 0.25 dB against that version. In the separate two-path cases, it loses about 0.82–0.93 dB to the same version.

The results support combining both types of input to improve the average under this training setup. They do not show that both are necessary in every channel. The reduced budget also does not replace the main study. All models store 188 parameters, but masked input weights have no effect. Removing noise or spectral inputs from the controller does not remove all related information: the measurement update still uses noise variance, and the filter still acts on the estimated channel.

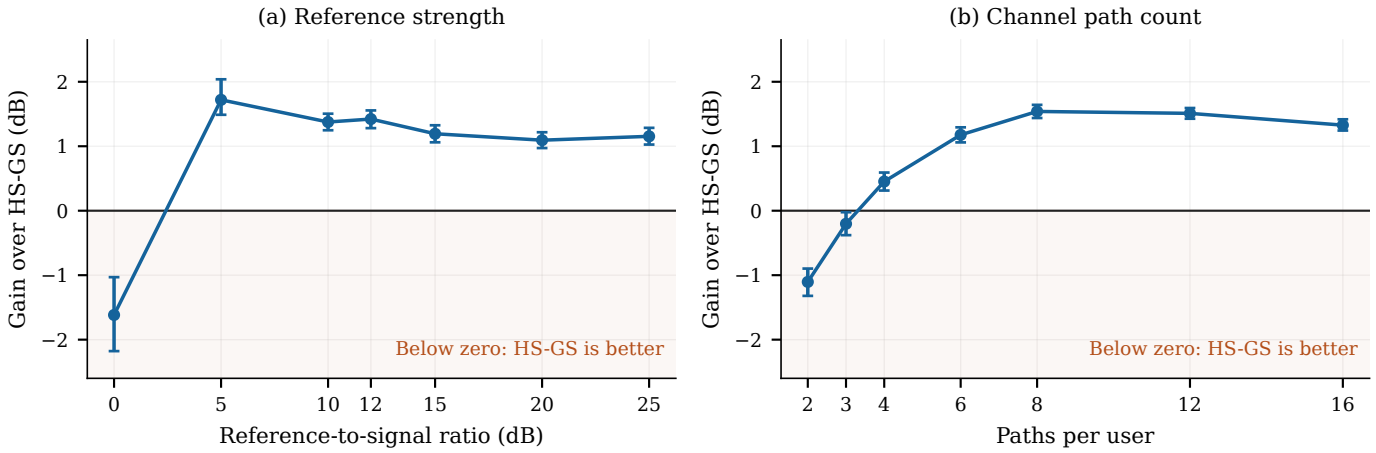


Fig. 4. Limits of the main method at SNR 5 dB. (a) Reference strength with random three-to-seven-path channels. (b) Fixed path counts at RSR 12 dB. Positive gain favors the proposed estimator; negative gain favors HS-GS. Error bars are pointwise paired-world 95% intervals conditional on the trained models. The path-count sweep overlaps training support.

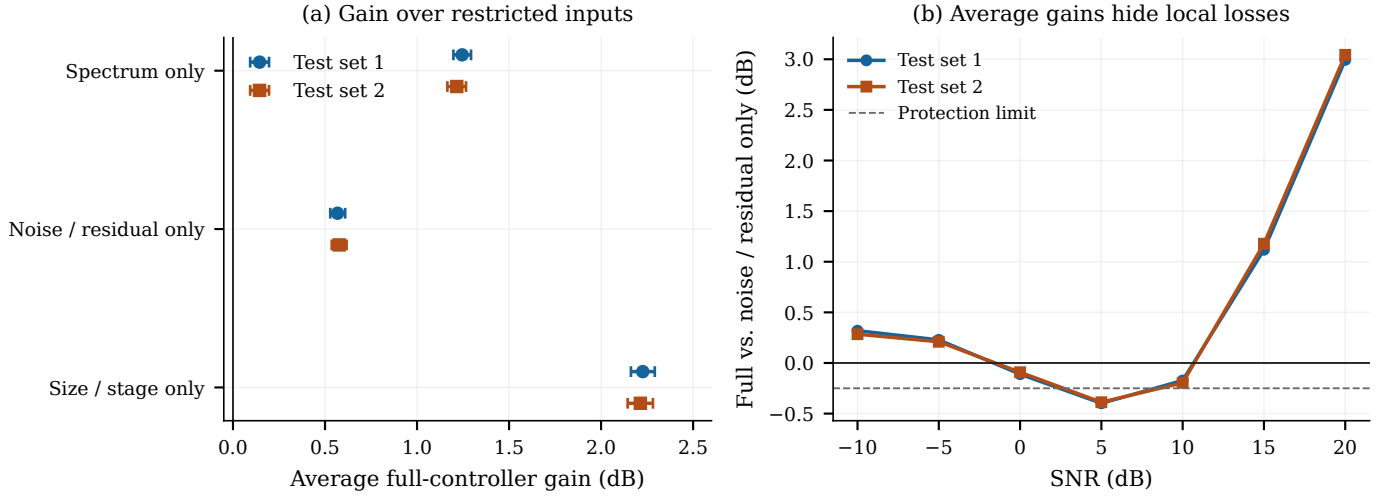


Fig. 5. Smaller-budget controller-input study. (a) Average gain of the full controller over versions with restricted inputs. Intervals adjust for three comparisons within each test set and are conditional on the trained models. (b) Full versus noise/residual-only inputs by SNR, showing the loss at 5 dB. The dashed line marks the predeclared 0.25 dB protection limit. Both fresh test sets are shown; each contains 128 cases per condition.

VI. Discussion and Conclusion

The useful result is not that Hankel filtering always helps. It is that a small model can learn how strongly to use this structure, without true-channel training targets, and provide a useful average accuracy and processing-time result in the tested setting. The input-removal study supports using both channel-structure and noise-related information, while its losses show why an average alone is insufficient. The practical use case is repeated estimation under conditions similar to the simulated training population, where offline training can be reused. This is an accuracy–processing-cost result for the implemented comparators, not a universal replacement for classical estimation.

All results are simulations. Reference calibration is assumed known, and no physical receiver measurements are used. Three seeds sharing training cases are not three independent datasets. We have not proved unique channel recovery in every setting or convergence of the full learned estimator. The comparisons also do not establish superiority over published URformer performance. Noise-dependent regularized classical controls and broader independent training/pilot replications would be needed for a stronger robustness claim. We do not infer the cause of a loss solely from its association with SNR, path count or reference strength.

Within these limits, learned Hankel filtering offers a compact, label-free option for magnitude-only channel estimation. Its gains and its weak-reference and few-path failures should be considered together when deciding whether to use it.

Appendix
Initialization and Numerical Checks

For array element n, form $A_n \in \mathbb{C}^{P\times(K+1)}$ with row $[s_p^T, b_{np}]$. Let u_n be the dominant eigenvector of $A_n^H \operatorname{diag}(z_n) A_n$ and set

$$c_n = \frac{\sum_p z_{np} |(A_n u_n)_p|}{\sum_p |(A_n u_n)_p|^2}. \tag{17}$$

The first K components of $c_n u_n \exp[-j \arg(u_{n,K+1})]$ give the initial channel row. During training, this uses only the 21 input pilots. This initializer is part of the shared estimation pipeline, not a new contribution.

Numerical checks cover the Rician likelihood and derivatives, complex noise variance, finite gradients, repeated singular values, pilot-split isolation, single/double-precision agreement and deterministic checkpoint recovery. The latest input study passed its saved-output audit for all 432 result files and 24 checkpoint replays. Such checks verify the implementation; they do not turn the failed protection criterion into a successful result.

One finite-input SVD failure occurred in the classical HS-GS array-size test ($N = 64$). The same matrix was recomputed using SciPy's alternative SVD driver; the recorded relative reconstruction error was below 3×10^{-15}. The trial was retained, and its inputs, equations and iteration budget were unchanged.

REPRODUCIBILITY

The accompanying reproducibility files include source code, frozen configurations, model checkpoints, compact per-case errors, timing records and source hashes. Recorded seeds specify the simulation datasets. Full observation arrays and full predicted channels are omitted from the compact package; this omission is listed explicitly. A small saved observation subset supports checkpoint replay. The numerical environment is Python 3.12 with NumPy 2.4.6, SciPy 1.17.1 and PyTorch 2.13.0.

ACKNOWLEDGMENT

OpenAI Codex [11] assisted with drafting and revising all sections, mathematical exposition, experiment and analysis code, and preparation of Figs. 1–5. Numerical plots were generated from saved simulation outputs. The author is responsible for the scientific claims, references and final submitted content.

REFERENCES

[1] B. Xu, J. Zhang, Z. Chen, B. Cheng, Z. Liu, Y.-C. Wu, and B. Ai, "Channel estimation for Rydberg atomic receivers," *IEEE Wireless Communications Letters*, vol. 14, no. 9, pp. 2957–2961, 2025, author manuscript: arXiv:2503.08985.

[2] J. Xiao, J. Wang, M. Zeng, H. Xu, X. Li, and A. Nallanathan, "Channel estimation for Rydberg atomic quantum receivers: Unrolled phase retrieval from holographic snapshots," *IEEE Signal Processing Letters*, vol. 33, pp. 1696–1700, 2026.

[3] B. Yaman, S. A. H. Hosseini, S. Moeller, J. Ellermann, K. Ugurbil, and M. Akcakaya, "Self-supervised learning of physics-guided reconstruction neural networks without fully sampled reference data," *Magnetic Resonance in Medicine*, vol. 84, no. 6, pp. 3172–3191, 2020.

[4] Z. Liu, Q. Ren, W. Yang, Y. Tong, H. Wang, Y. Zhang, R. Zhi, J. Xie, M. Jing, H. Zhang, L. Xiao, S. Jia, K. Tang, and L. Zhang, "Self-supervised learning with noisy dataset for Rydberg microwave sensors denoising," arXiv:2601.01924, 2026.

[5] Y. Huang, J. Zhao, Z. Wang, V. Orekhov, D. Guo, and X. Qu, "Exponential signal reconstruction with deep Hankel matrix factorization," arXiv:2007.06246v4, 2021, accepted author manuscript, IEEE Transactions on Neural Networks and Learning Systems.

[6] C. Luo, H. Wang, T. Xie, Q. Jin, G. Chen, Z.-X. Cui, and D. Liang, "Matrix completion-informed deep unfolded equilibrium models for self-supervised k-space interpolation in MRI," arXiv:2309.13571, 2023.

[7] V. Saragadam, R. Balestriero, A. Veeraraghavan, and R. G. Baraniuk, "DeepTensor: Low-rank tensor decomposition with deep network priors," arXiv:2204.03145, 2022.

[8] C. S. Parker, A. Schroder, S. C. Epstein, J. Cole, D. C. Alexander, and H. Zhang, "Rician likelihood loss for quantitative MRI using self-supervised deep learning," arXiv:2307.07072, 2023.

[9] A. Haatim, "Performance analysis of a Hankel-structured channel estimator for Rydberg atomic MIMO receivers with magnitude-only observations," 2026, unpublished companion manuscript.

[10] CCAI, "2026 the 6th international conference on computer communication and artificial intelligence: Conference program," 2026, p. 22, poster P14. Accessed: Sep. 23, 2026. [Online]. Available: https://ccai.net/CCAI2026-Program.pdf

[11] OpenAI, "Codex," 2026, aI assistance used in September 2026. [Online]. Available: https://openai.com/codex/